Retail Sales Analytics using Oracle SQL

Complete Database Project

```
# Retail-Sales-Analytics-Oracle-SQL


-- ====================================
-- RETAIL SALES DATABASE
-- CREATE TABLES (Oracle SQL)
-- ====================================

----------------------------------------
-- 1. CATEGORY
----------------------------------------
CREATE TABLE Category (
    Category_ID NUMBER PRIMARY KEY,
    Category_Name VARCHAR2(50),
    Description VARCHAR2(100)
);

----------------------------------------
-- 2. CUSTOMER
----------------------------------------
CREATE TABLE Customer (
    Customer_ID NUMBER PRIMARY KEY,
    Customer_Name VARCHAR2(50),
    Gender VARCHAR2(10),
    City VARCHAR2(30),
    Phone VARCHAR2(15),
    Member_Type VARCHAR2(20),
    Points NUMBER
);

----------------------------------------
-- 3. PRODUCT
----------------------------------------
CREATE TABLE Product (
    Product_ID NUMBER PRIMARY KEY,
    Product_Name VARCHAR2(50),
    Category_ID NUMBER,
    Brand VARCHAR2(30),
    Price NUMBER(10,2),
    Stock NUMBER,
    Rating NUMBER(2,1),
    CONSTRAINT FK_Product_Category
    FOREIGN KEY (Category_ID)
    REFERENCES Category(Category_ID)
);

----------------------------------------
-- 4. EMPLOYEE
----------------------------------------
CREATE TABLE Employee (
    Employee_ID NUMBER PRIMARY KEY,
    Employee_Name VARCHAR2(50),
    Department VARCHAR2(30),
    Salary NUMBER(10,2),
    Hire_Date DATE,
    Gender VARCHAR2(10),
    Status VARCHAR2(20)
);

----------------------------------------
-- 5. ORDERS
----------------------------------------
CREATE TABLE Orders (
    Order_ID NUMBER PRIMARY KEY,
    Customer_ID NUMBER,
    Employee_ID NUMBER,
    Order_Date DATE,
    Total_Amount NUMBER(10,2),
```

```
    Payment_Type VARCHAR2(20),
    Status VARCHAR2(20),
    CONSTRAINT FK_Order_Customer
    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID),
    CONSTRAINT FK_Order_Employee
    FOREIGN KEY (Employee_ID)
    REFERENCES Employee(Employee_ID)
);

----------------------------------------
-- 6. ORDER_DETAILS
----------------------------------------
CREATE TABLE Order_Details (
    Detail_ID NUMBER PRIMARY KEY,
    Order_ID NUMBER,
    Product_ID NUMBER,
    Quantity NUMBER,
    Price NUMBER(10,2),
    Discount NUMBER(5,2),
    Total NUMBER(10,2),
    CONSTRAINT FK_Detail_Order
    FOREIGN KEY (Order_ID)
    REFERENCES Orders(Order_ID),
    CONSTRAINT FK_Detail_Product
    FOREIGN KEY (Product_ID)
    REFERENCES Product(Product_ID)
);

----------------------------------------
-- 7. PAYMENT
----------------------------------------
CREATE TABLE Payment (
    Payment_ID NUMBER PRIMARY KEY,
    Order_ID NUMBER,
    Amount NUMBER(10,2),
    Payment_Method VARCHAR2(20),
    Payment_Date DATE,
    Status VARCHAR2(20),
    Transaction_ID VARCHAR2(30),
    CONSTRAINT FK_Payment_Order
    FOREIGN KEY (Order_ID)
    REFERENCES Orders(Order_ID)
);

----------------------------------------
-- 8. RETURN
----------------------------------------
CREATE TABLE Return_Order (
    Return_ID NUMBER PRIMARY KEY,
    Order_ID NUMBER,
    Product_ID NUMBER,
    Reason VARCHAR2(100),
    Refund_Amount NUMBER(10,2),
    Return_Date DATE,
    Status VARCHAR2(20),
    CONSTRAINT FK_Return_Order
    FOREIGN KEY (Order_ID)
    REFERENCES Orders(Order_ID),
    CONSTRAINT FK_Return_Product
    FOREIGN KEY (Product_ID)
    REFERENCES Product(Product_ID)
);


-- ====================================
-- RETAIL SALES DATABASE
-- INSERT RECORDS (Oracle SQL)
-- ====================================

----------------------------------------
--1.Insert Data into Category Table
----------------------------------------
INSERT ALL
INTO Category VALUES (1,'Electronics','Electronic Items')
INTO Category VALUES (2,'Clothing','Men and Women Wear')
INTO Category VALUES (3,'Groceries','Daily Essentials')
```

```
INTO Category VALUES (4,'Furniture','Home Furniture')
INTO Category VALUES (5,'Sports','Sports Items')
INTO Category VALUES (6,'Books','Books and Stationery')
INTO Category VALUES (7,'Beauty','Beauty Products')
INTO Category VALUES (8,'Toys','Kids Toys')
INTO Category VALUES (9,'Footwear','Shoes and Sandals')
INTO Category VALUES (10,'Accessories','Fashion Accessories')
SELECT * FROM Dual;

COMMIT;



----------------------------------------
--2.Insert Data into Customer Table
----------------------------------------
INSERT ALL
INTO Customer VALUES (101,'Rahul','Male','Hyderabad','9876543210','Gold',500)
INTO Customer VALUES (102,'Priya','Female','Bengaluru','9876543211','Silver',350)
INTO Customer VALUES (103,'Amit','Male','Chennai','9876543212','Gold',700)
INTO Customer VALUES (104,'Sneha','Female','Mumbai','9876543213','Bronze',200)
INTO Customer VALUES (105,'Kiran','Male','Pune','9876543214','Silver',450)
INTO Customer VALUES (106,'Anjali','Female','Delhi','9876543215','Gold',800)
INTO Customer VALUES (107,'Rohit','Male','Kolkata','9876543216','Bronze',150)
INTO Customer VALUES (108,'Meena','Female','Vijayawada','9876543217','Silver',400)
INTO Customer VALUES (109,'Arjun','Male','Visakhapatnam','9876543218','Gold',650)
INTO Customer VALUES (110,'Divya','Female','Mysuru','9876543219','Silver',300)
SELECT * FROM Dual;

COMMIT;



----------------------------------------
--3.Insert Data into Product Table
----------------------------------------
INSERT ALL
INTO Product VALUES (1001,'Laptop',1,'Dell',55000,20,4.8)
INTO Product VALUES (1002,'Smartphone',1,'Samsung',25000,35,4.6)
INTO Product VALUES (1003,'T-Shirt',2,'Nike',1200,50,4.3)
INTO Product VALUES (1004,'Rice Bag',3,'India Gate',1800,40,4.5)
INTO Product VALUES (1005,'Sofa',4,'Godrej',30000,10,4.4)
INTO Product VALUES (1006,'Cricket Bat',5,'SS',3500,25,4.7)
INTO Product VALUES (1007,'Notebook',6,'Classmate',80,200,4.2)
INTO Product VALUES (1008,'Face Cream',7,'Lakme',450,60,4.1)
INTO Product VALUES (1009,'Toy Car',8,'Funskool',700,45,4.3)
INTO Product VALUES (1010,'Running Shoes',9,'Adidas',4500,30,4.9)
SELECT * FROM Dual;

COMMIT;



----------------------------------------
--4.Insert Data into Employee Table
----------------------------------------
INSERT ALL
INTO Employee VALUES (201,'Ramesh','Sales',35000,TO_DATE('10-01-2022','DD-MM-YYYY'),'Male','Active')
INTO Employee VALUES (202,'Priya','Sales',38000,TO_DATE('15-03-2021','DD-MM-YYYY'),'Female','Active'
INTO Employee VALUES (203,'Arun','Marketing',42000,TO_DATE('20-07-2020','DD-MM-YYYY'),'Male','Active
INTO Employee VALUES (204,'Sneha','HR',40000,TO_DATE('05-11-2021','DD-MM-YYYY'),'Female','Active')
INTO Employee VALUES (205,'Kiran','Finance',45000,TO_DATE('18-08-2019','DD-MM-YYYY'),'Male','Active'
INTO Employee VALUES (206,'Divya','Sales',37000,TO_DATE('12-02-2023','DD-MM-YYYY'),'Female','Active'
INTO Employee VALUES (207,'Ravi','IT',50000,TO_DATE('25-09-2018','DD-MM-YYYY'),'Male','Active')
INTO Employee VALUES (208,'Anjali','HR',39000,TO_DATE('30-06-2022','DD-MM-YYYY'),'Female','Active')
INTO Employee VALUES (209,'Vijay','Finance',47000,TO_DATE('14-12-2020','DD-MM-YYYY'),'Male','Active'
INTO Employee VALUES (210,'Meena','Marketing',41000,TO_DATE('08-05-2021','DD-MM-YYYY'),'Female','Acti
SELECT * FROM Dual;

COMMIT;



----------------------------------------
--5.Insert Data into Orders Table
----------------------------------------
```

```
INSERT ALL
INTO Orders VALUES (5001,101,201,TO_DATE('01-06-2025','DD-MM-YYYY'),55000,'UPI','Delivered')
INTO Orders VALUES (5002,102,202,TO_DATE('03-06-2025','DD-MM-YYYY'),25000,'Card','Delivered')
INTO Orders VALUES (5003,103,203,TO_DATE('05-06-2025','DD-MM-YYYY'),1200,'Cash','Delivered')
INTO Orders VALUES (5004,104,204,TO_DATE('07-06-2025','DD-MM-YYYY'),1800,'UPI','Delivered')
INTO Orders VALUES (5005,105,205,TO_DATE('10-06-2025','DD-MM-YYYY'),30000,'Card','Pending')
INTO Orders VALUES (5006,106,206,TO_DATE('12-06-2025','DD-MM-YYYY'),3500,'Cash','Delivered')
INTO Orders VALUES (5007,107,207,TO_DATE('15-06-2025','DD-MM-YYYY'),80,'UPI','Delivered')
INTO Orders VALUES (5008,108,208,TO_DATE('18-06-2025','DD-MM-YYYY'),450,'Card','Pending')
INTO Orders VALUES (5009,109,209,TO_DATE('20-06-2025','DD-MM-YYYY'),700,'Cash','Delivered')
INTO Orders VALUES (5010,110,210,TO_DATE('22-06-2025','DD-MM-YYYY'),4500,'UPI','Delivered')
SELECT * FROM Dual;

COMMIT;




----------------------------------------
--6.Insert Data into Order_Details Table
----------------------------------------
INSERT ALL
INTO Order_Details VALUES (1,5001,1001,1,55000,0,55000)
INTO Order_Details VALUES (2,5002,1002,1,25000,500,24500)
INTO Order_Details VALUES (3,5003,1003,2,1200,100,2300)
INTO Order_Details VALUES (4,5004,1004,1,1800,0,1800)
INTO Order_Details VALUES (5,5005,1005,1,30000,500,29500)
INTO Order_Details VALUES (6,5006,1006,1,3500,200,3300)
INTO Order_Details VALUES (7,5007,1007,5,80,0,400)
INTO Order_Details VALUES (8,5008,1008,2,450,50,850)
INTO Order_Details VALUES (9,5009,1009,1,700,0,700)
INTO Order_Details VALUES (10,5010,1010,1,4500,300,4200)
SELECT * FROM Dual;

COMMIT;




----------------------------------------
--7.Insert Data into Payment Table
----------------------------------------
INSERT ALL
INTO Payment VALUES (1,5001,55000,'UPI',TO_DATE('01-06-2025','DD-MM-YYYY'),'Success','TXN1001')
INTO Payment VALUES (2,5002,25000,'Card',TO_DATE('03-06-2025','DD-MM-YYYY'),'Success','TXN1002')
INTO Payment VALUES (3,5003,1200,'Cash',TO_DATE('05-06-2025','DD-MM-YYYY'),'Success','TXN1003')
INTO Payment VALUES (4,5004,1800,'UPI',TO_DATE('07-06-2025','DD-MM-YYYY'),'Success','TXN1004')
INTO Payment VALUES (5,5005,30000,'Card',TO_DATE('10-06-2025','DD-MM-YYYY'),'Pending','TXN1005')
INTO Payment VALUES (6,5006,3500,'Cash',TO_DATE('12-06-2025','DD-MM-YYYY'),'Success','TXN1006')
INTO Payment VALUES (7,5007,80,'UPI',TO_DATE('15-06-2025','DD-MM-YYYY'),'Success','TXN1007')
INTO Payment VALUES (8,5008,450,'Card',TO_DATE('18-06-2025','DD-MM-YYYY'),'Pending','TXN1008')
INTO Payment VALUES (9,5009,700,'Cash',TO_DATE('20-06-2025','DD-MM-YYYY'),'Success','TXN1009')
INTO Payment VALUES (10,5010,4500,'UPI',TO_DATE('22-06-2025','DD-MM-YYYY'),'Success','TXN1010')
SELECT * FROM Dual;

COMMIT;




----------------------------------------
--8.Insert Data into Return_Orde Table
----------------------------------------
INSERT ALL
INTO Return_Order VALUES (1,5001,1001,'Damaged',55000,TO_DATE('05-06-2025','DD-MM-YYYY'),'Approved')
INTO Return_Order VALUES (2,5002,1002,'Wrong Product',25000,TO_DATE('08-06-2025','DD-MM-YYYY'),'Appro
INTO Return_Order VALUES (3,5003,1003,'Size Issue',1200,TO_DATE('10-06-2025','DD-MM-YYYY'),'Pending'
INTO Return_Order VALUES (4,5004,1004,'Expired Product',1800,TO_DATE('12-06-2025','DD-MM-YYYY'),'App
INTO Return_Order VALUES (5,5005,1005,'Damaged',30000,TO_DATE('15-06-2025','DD-MM-YYYY'),'Rejected')
INTO Return_Order VALUES (6,5006,1006,'Wrong Item',3500,TO_DATE('18-06-2025','DD-MM-YYYY'),'Approved
INTO Return_Order VALUES (7,5007,1007,'Poor Quality',400,TO_DATE('20-06-2025','DD-MM-YYYY'),'Pending
INTO Return_Order VALUES (8,5008,1008,'Allergy',850,TO_DATE('22-06-2025','DD-MM-YYYY'),'Approved')
INTO Return_Order VALUES (9,5009,1009,'Broken',700,TO_DATE('24-06-2025','DD-MM-YYYY'),'Approved')
INTO Return_Order VALUES (10,5010,1010,'Size Issue',4200,TO_DATE('26-06-2025','DD-MM-YYYY'),'Pending
SELECT * FROM Dual;
COMMIT;
```

```
-- ============================================================
-- SELECT QUERIES (1-15)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-----------------------------------------------------
-- 1. Display all customer details.
-----------------------------------------------------
SELECT *
FROM Customer;

-----------------------------------------------------
-- 2. Display all product details.
-----------------------------------------------------
SELECT *
FROM Product;

-----------------------------------------------------
-- 3. Display all employee details.
-----------------------------------------------------
SELECT *
FROM Employee;

-----------------------------------------------------
-- 4. Display all order details.
-----------------------------------------------------
SELECT *
FROM Orders;

-----------------------------------------------------
-- 5. Display all payment details.
-----------------------------------------------------
SELECT *
FROM Payment;

-----------------------------------------------------
-- 6. Display all returned orders.
-----------------------------------------------------
SELECT *
FROM Return_Order;

-----------------------------------------------------
-- 7. Display all category details.
-----------------------------------------------------
SELECT *
FROM Category;

-----------------------------------------------------
-- 8. Display customer names only.
-----------------------------------------------------
SELECT Customer_Name
FROM Customer;

-----------------------------------------------------
-- 9. Display product name and price.
-----------------------------------------------------
SELECT Product_Name,
       Price
FROM Product;

-----------------------------------------------------
-- 10. Display employee name and department.
-----------------------------------------------------
SELECT Employee_Name,
       Department
FROM Employee;

-----------------------------------------------------
-- 11. Display order ID and total amount.
-----------------------------------------------------
SELECT Order_ID,
       Total_Amount
FROM Orders;

-----------------------------------------------------
-- 12. Display payment method and amount.
```

```
-------------------------------------------------
SELECT Payment_Method,
       Amount
FROM Payment;

-------------------------------------------------
-- 13. Display product name and stock.
-------------------------------------------------
SELECT Product_Name,
       Stock
FROM Product;

-------------------------------------------------
-- 14. Display customer name and city.
-------------------------------------------------
SELECT Customer_Name,
       City
FROM Customer;

-------------------------------------------------
-- 15. Display category name and description.
-------------------------------------------------
SELECT Category_Name,
       Description
FROM Category;



-- ============================================================
-- WHERE CLAUSE (16-30)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-------------------------------------------------
-- 16. Display products with price greater than 5000.
-------------------------------------------------
SELECT *
FROM Product
WHERE Price > 5000;

-------------------------------------------------
-- 17. Display customers from Hyderabad.
-------------------------------------------------
SELECT *
FROM Customer
WHERE City = 'Hyderabad';

-------------------------------------------------
-- 18. Display female customers.
-------------------------------------------------
SELECT *
FROM Customer
WHERE Gender = 'Female';

-------------------------------------------------
-- 19. Display active employees.
-------------------------------------------------
SELECT *
FROM Employee
WHERE Status = 'Active';

-------------------------------------------------
-- 20. Display delivered orders.
-------------------------------------------------
SELECT *
FROM Orders
WHERE Order_Status = 'Delivered';

-------------------------------------------------
-- 21. Display pending orders.
-------------------------------------------------
SELECT *
FROM Orders
WHERE Order_Status = 'Pending';

-------------------------------------------------
```

```
-- 22. Display products with rating greater than 4.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Rating > 4;

-----------------------------------------------------
-- 23. Display Gold members.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Member_Type = 'Gold';

-----------------------------------------------------
-- 24. Display employees earning more than 40000.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Salary > 40000;

-----------------------------------------------------
-- 25. Display products with stock less than 30.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Stock < 30;

-----------------------------------------------------
-- 26. Display products with price less than 1000.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Price < 1000;

-----------------------------------------------------
-- 27. Display customers whose points are greater than 500.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Points > 500;

-----------------------------------------------------
-- 28. Display products with stock greater than 50.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Stock > 50;

-----------------------------------------------------
-- 29. Display employees from Sales department.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Department = 'Sales';

-----------------------------------------------------
-- 30. Display successful payments.
-----------------------------------------------------
SELECT *
FROM Payment
WHERE Status = 'Success';



-- ============================================================
-- ORDER BY, DISTINCT, LIKE, BETWEEN, IN (31-45)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-----------------------------------------------------
-- 31. Display products in ascending order of price.
-----------------------------------------------------
SELECT *
FROM Product
ORDER BY Price ASC;
-----------------------------------------------------
```

```
-- 32. Display products in descending order of price.
-----------------------------------------------------
SELECT *
FROM Product
ORDER BY Price DESC;

-----------------------------------------------------
-- 33. Display customers in alphabetical order.
-----------------------------------------------------
SELECT *
FROM Customer
ORDER BY Customer_Name ASC;

-----------------------------------------------------
-- 34. Display employees by salary (highest first).
-----------------------------------------------------
SELECT *
FROM Employee
ORDER BY Salary DESC;

-----------------------------------------------------
-- 35. Display orders by order date.
-----------------------------------------------------
SELECT *
FROM Orders
ORDER BY Order_Date DESC;

-----------------------------------------------------
-- 36. Display distinct customer cities.
-----------------------------------------------------
SELECT DISTINCT City
FROM Customer;

-----------------------------------------------------
-- 37. Display distinct payment methods.
-----------------------------------------------------
SELECT DISTINCT Payment_Method
FROM Payment;

-----------------------------------------------------
-- 38. Display customers whose name starts with 'R'.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Customer_Name LIKE 'R%';

-----------------------------------------------------
-- 39. Display customers whose name ends with 'a'.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Customer_Name LIKE '%a';

-----------------------------------------------------
-- 40. Display products whose name contains 'Phone'.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Product_Name LIKE '%Phone%';

-----------------------------------------------------
-- 41. Display products priced between 1000 and 10000.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Price BETWEEN 1000 AND 10000;

-----------------------------------------------------
-- 42. Display employees whose salary is between
-- 30000 and 60000.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Salary BETWEEN 30000 AND 60000;

-----------------------------------------------------
-- 43. Display customers from Hyderabad or Bengaluru.
```

```
-------------------------------------------------
SELECT *
FROM Customer
WHERE City IN ('Hyderabad','Bengaluru');

-------------------------------------------------
-- 44. Display payments made by UPI or Card.
-------------------------------------------------
SELECT *
FROM Payment
WHERE Payment_Method IN ('UPI','Card');

-------------------------------------------------
-- 45. Display products belonging to Category 1 or 2.
-------------------------------------------------
SELECT *
FROM Product
WHERE Category_ID IN (1,2);



-- ============================================================
-- AGGREGATE FUNCTIONS, GROUP BY, HAVING (46-60)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-------------------------------------------------
-- 46. Count the total number of customers.
-------------------------------------------------
SELECT COUNT(*) AS Total_Customers
FROM Customer;

-------------------------------------------------
-- 47. Count the total number of products.
-------------------------------------------------
SELECT COUNT(*) AS Total_Products
FROM Product;

-------------------------------------------------
-- 48. Count the total number of employees.
-------------------------------------------------
SELECT COUNT(*) AS Total_Employees
FROM Employee;

-------------------------------------------------
-- 49. Find the total sales amount.
-------------------------------------------------
SELECT SUM(Total_Amount) AS Total_Sales
FROM Orders;

-------------------------------------------------
-- 50. Find the average product price.
-------------------------------------------------
SELECT AVG(Price) AS Average_Price
FROM Product;

-------------------------------------------------
-- 51. Find the highest product price.
-------------------------------------------------
SELECT MAX(Price) AS Highest_Price
FROM Product;

-------------------------------------------------
-- 52. Find the lowest product price.
-------------------------------------------------
SELECT MIN(Price) AS Lowest_Price
FROM Product;

-------------------------------------------------
-- 53. Find the highest employee salary.
-------------------------------------------------
SELECT MAX(Salary) AS Highest_Salary
FROM Employee;

-------------------------------------------------
```

```sql
-- 54. Find the lowest employee salary.
-----------------------------------------------------
SELECT MIN(Salary) AS Lowest_Salary
FROM Employee;

-----------------------------------------------------
-- 55. Find the average employee salary.
-----------------------------------------------------
SELECT AVG(Salary) AS Average_Salary
FROM Employee;

-----------------------------------------------------
-- 56. Display the number of customers in each city.
-----------------------------------------------------
SELECT City,
       COUNT(*) AS Total_Customers
FROM Customer
GROUP BY City;

-----------------------------------------------------
-- 57. Display the number of employees in each department.
-----------------------------------------------------
SELECT Department,
       COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Department;

-----------------------------------------------------
-- 58. Display the total salary by department.
-----------------------------------------------------
SELECT Department,
       SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department;

-----------------------------------------------------
-- 59. Display departments having more than one employee.
-----------------------------------------------------
SELECT Department,
       COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Department
HAVING COUNT(*) > 1;

-----------------------------------------------------
-- 60. Display cities having more than one customer.
-----------------------------------------------------
SELECT City,
       COUNT(*) AS Total_Customers
FROM Customer
GROUP BY City
HAVING COUNT(*) > 1;




-- ============================================================
-- JOIN QUERIES (61-80)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-----------------------------------------------------
-- 61. Display customer name and order ID.
-----------------------------------------------------
SELECT C.Customer_Name,
       O.Order_ID
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID;

-----------------------------------------------------
-- 62. Display customer name and total order amount.
-----------------------------------------------------
SELECT C.Customer_Name,
       O.Total_Amount
FROM Customer C
```

```
JOIN Orders O
ON C.Customer_ID = O.Customer_ID;

-----------------------------------------------------
-- 63. Display order ID and payment method.
-----------------------------------------------------
SELECT O.Order_ID,
       P.Payment_Method
FROM Orders O
JOIN Payment P
ON O.Order_ID = P.Order_ID;

-----------------------------------------------------
-- 64. Display order ID and payment status.
-----------------------------------------------------
SELECT O.Order_ID,
       P.Status
FROM Orders O
JOIN Payment P
ON O.Order_ID = P.Order_ID;

-----------------------------------------------------
-- 65. Display employee name and order ID.
-----------------------------------------------------
SELECT E.Employee_Name,
       O.Order_ID
FROM Employee E
JOIN Orders O
ON E.Employee_ID = O.Employee_ID;

-----------------------------------------------------
-- 66. Display product name and category name.
-----------------------------------------------------
SELECT P.Product_Name,
       C.Category_Name
FROM Product P
JOIN Category C
ON P.Category_ID = C.Category_ID;

-----------------------------------------------------
-- 67. Display product name and quantity ordered.
-----------------------------------------------------
SELECT P.Product_Name,
       OD.Quantity
FROM Product P
JOIN Order_Details OD
ON P.Product_ID = OD.Product_ID;

-----------------------------------------------------
-- 68. Display order ID, product name and quantity.
-----------------------------------------------------
SELECT O.Order_ID,
       P.Product_Name,
       OD.Quantity
FROM Orders O
JOIN Order_Details OD
ON O.Order_ID = OD.Order_ID
JOIN Product P
ON OD.Product_ID = P.Product_ID;

-----------------------------------------------------
-- 69. Display customer name and payment method.
-----------------------------------------------------
SELECT C.Customer_Name,
       P.Payment_Method
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Payment P
ON O.Order_ID = P.Order_ID;

-----------------------------------------------------
-- 70. Display customer name and employee name.
-----------------------------------------------------
SELECT C.Customer_Name,
       E.Employee_Name
FROM Customer C
```

```
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Employee E
ON O.Employee_ID = E.Employee_ID;

-----------------------------------------------------
-- 71. Display customer name, order amount and payment method.
-----------------------------------------------------
SELECT C.Customer_Name,
       O.Total_Amount,
       P.Payment_Method
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Payment P
ON O.Order_ID = P.Order_ID;

-----------------------------------------------------
-- 72. Display product name and refund amount.
-----------------------------------------------------
SELECT P.Product_Name,
       R.Refund_Amount
FROM Product P
JOIN Return_Order R
ON P.Product_ID = R.Product_ID;

-----------------------------------------------------
-- 73. Display customer name and returned product.
-----------------------------------------------------
SELECT C.Customer_Name,
       P.Product_Name
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Return_Order R
ON O.Order_ID = R.Order_ID
JOIN Product P
ON R.Product_ID = P.Product_ID;

-----------------------------------------------------
-- 74. Display employee name and total order amount.
-----------------------------------------------------
SELECT E.Employee_Name,
       O.Total_Amount
FROM Employee E
JOIN Orders O
ON E.Employee_ID = O.Employee_ID;

-----------------------------------------------------
-- 75. Display category name and product price.
-----------------------------------------------------
SELECT C.Category_Name,
       P.Price
FROM Category C
JOIN Product P
ON C.Category_ID = P.Category_ID;

-----------------------------------------------------
-- 76. Display order ID and return status.
-----------------------------------------------------
SELECT O.Order_ID,
       R.Status
FROM Orders O
JOIN Return_Order R
ON O.Order_ID = R.Order_ID;

-----------------------------------------------------
-- 77. Display customer name, city and total order amount.
-----------------------------------------------------
SELECT C.Customer_Name,
       C.City,
       O.Total_Amount
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID;

-----------------------------------------------------
```

```sql
-- 78. Display product name, price and stock.
-------------------------------------------------
SELECT Product_Name,
       Price,
       Stock
FROM Product;

-------------------------------------------------
-- 79. Display employee name, department and salary.
-------------------------------------------------
SELECT Employee_Name,
       Department,
       Salary
FROM Employee;

-------------------------------------------------
-- 80. Display customer name, product name and quantity ordered.
-------------------------------------------------
SELECT C.Customer_Name,
       P.Product_Name,
       OD.Quantity
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Order_Details OD
ON O.Order_ID = OD.Order_ID
JOIN Product P
ON OD.Product_ID = P.Product_ID;



-- ============================================================
-- SUBQUERIES, SET OPERATORS & STRING FUNCTIONS (81-100)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-------------------------------------------------
-- 81. Display products with price greater than the average price.
-------------------------------------------------
SELECT *
FROM Product
WHERE Price >
(
    SELECT AVG(Price)
    FROM Product
);

-------------------------------------------------
-- 82. Display employees earning more than the average salary.
-------------------------------------------------
SELECT *
FROM Employee
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM Employee
);

-------------------------------------------------
-- 83. Display customers who have placed orders.
-------------------------------------------------
SELECT *
FROM Customer
WHERE Customer_ID IN
(
    SELECT Customer_ID
    FROM Orders
);

-------------------------------------------------
-- 84. Display customers who have not placed any orders.
-------------------------------------------------
SELECT *
FROM Customer
WHERE Customer_ID NOT IN
```

```
(
    SELECT Customer_ID
    FROM Orders
);

-----------------------------------------------------
-- 85. Display the product with the highest price.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Price =
(
    SELECT MAX(Price)
    FROM Product
);

-----------------------------------------------------
-- 86. Display the employee with the highest salary.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Salary =
(
    SELECT MAX(Salary)
    FROM Employee
);

-----------------------------------------------------
-- 87. Display the customer with the highest loyalty points.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Points =
(
    SELECT MAX(Points)
    FROM Customer
);

-----------------------------------------------------
-- 88. Display products with stock less than the average stock.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Stock <
(
    SELECT AVG(Stock)
    FROM Product
);

-----------------------------------------------------
-- 89. Display orders with amount greater than average.
-----------------------------------------------------
SELECT *
FROM Orders
WHERE Total_Amount >
(
    SELECT AVG(Total_Amount)
    FROM Orders
);

-----------------------------------------------------
-- 90. Display employees earning less than the average salary.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Salary <
(
    SELECT AVG(Salary)
    FROM Employee
);

-----------------------------------------------------
-- 91. Display all customer names and employee names.
-----------------------------------------------------
SELECT Customer_Name AS Name
FROM Customer
UNION
```

```
SELECT Employee_Name
FROM Employee;

-----------------------------------------------------
-- 92. Display all customer and employee names
-- including duplicates.
-----------------------------------------------------
SELECT Customer_Name AS Name
FROM Customer
UNION ALL
SELECT Employee_Name
FROM Employee;

-----------------------------------------------------
-- 93. Display common city names.
-----------------------------------------------------
SELECT City
FROM Customer
INTERSECT
SELECT City
FROM Customer;

-----------------------------------------------------
-- 94. Display customer names that are not employee names.
-----------------------------------------------------
SELECT Customer_Name
FROM Customer
MINUS
SELECT Employee_Name
FROM Employee;

-----------------------------------------------------
-- 95. Display employee names that are not customer names.
-----------------------------------------------------
SELECT Employee_Name
FROM Employee
MINUS
SELECT Customer_Name
FROM Customer;

-----------------------------------------------------
-- 96. Display customer names in uppercase.
-----------------------------------------------------
SELECT UPPER(Customer_Name)
FROM Customer;

-----------------------------------------------------
-- 97. Display product names in lowercase.
-----------------------------------------------------
SELECT LOWER(Product_Name)
FROM Product;

-----------------------------------------------------
-- 98. Display employee name and length of name.
-----------------------------------------------------
SELECT Employee_Name,
       LENGTH(Employee_Name) AS Name_Length
FROM Employee;

-----------------------------------------------------
-- 99. Display first three characters of each product name.
-----------------------------------------------------
SELECT Product_Name,
       SUBSTR(Product_Name,1,3) AS Short_Name
FROM Product;

-----------------------------------------------------
-- 100. Remove leading and trailing spaces from customer names.
-----------------------------------------------------
SELECT TRIM(Customer_Name)
FROM Customer;



-- ============================================================
-- NUMBER FUNCTIONS, DATE FUNCTIONS,
-- WINDOW FUNCTIONS & TRANSACTIONS (101-120)
```

```
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-----------------------------------------------------
-- 101. Round product prices.
-----------------------------------------------------
SELECT Product_Name,
       ROUND(Price) AS Rounded_Price
FROM Product;

-----------------------------------------------------
-- 102. Find square root of employee salaries.
-----------------------------------------------------
SELECT Employee_Name,
       SQRT(Salary) AS Salary_SQRT
FROM Employee;

-----------------------------------------------------
-- 103. Find remainder when stock is divided by 5.
-----------------------------------------------------
SELECT Product_Name,
       MOD(Stock,5) AS Remainder
FROM Product;

-----------------------------------------------------
-- 104. Find absolute value of -250.
-----------------------------------------------------
SELECT ABS(-250) AS Absolute_Value
FROM Dual;

-----------------------------------------------------
-- 105. Round average product price.
-----------------------------------------------------
SELECT ROUND(AVG(Price),2) AS Average_Price
FROM Product;

-----------------------------------------------------
-- 106. Display current system date.
-----------------------------------------------------
SELECT SYSDATE
FROM Dual;

-----------------------------------------------------
-- 107. Display employee joining year.
-----------------------------------------------------
SELECT Employee_Name,
       EXTRACT(YEAR FROM Hire_Date) AS Joining_Year
FROM Employee;

-----------------------------------------------------
-- 108. Display employee joining month.
-----------------------------------------------------
SELECT Employee_Name,
       EXTRACT(MONTH FROM Hire_Date) AS Joining_Month
FROM Employee;

-----------------------------------------------------
-- 109. Find number of working days.
-----------------------------------------------------
SELECT Employee_Name,
       SYSDATE - Hire_Date AS Working_Days
FROM Employee;

-----------------------------------------------------
-- 110. Add 30 days to current date.
-----------------------------------------------------
SELECT SYSDATE + 30 AS Future_Date
FROM Dual;

-----------------------------------------------------
-- 111. Rank employees by salary.
-----------------------------------------------------
SELECT Employee_Name,
       Salary,
       RANK() OVER(ORDER BY Salary DESC) AS Rank_No
FROM Employee;
```

```sql
-------------------------------------------------
-- 112. Dense Rank employees by salary.
-------------------------------------------------
SELECT Employee_Name,
       Salary,
       DENSE_RANK() OVER(ORDER BY Salary DESC) AS Dense_Rank
FROM Employee;

-------------------------------------------------
-- 113. Assign row numbers to products.
-------------------------------------------------
SELECT Product_Name,
       Price,
       ROW_NUMBER() OVER(ORDER BY Price DESC) AS Row_No
FROM Product;

-------------------------------------------------
-- 114. Rank products by rating.
-------------------------------------------------
SELECT Product_Name,
       Rating,
       RANK() OVER(ORDER BY Rating DESC) AS Rank_No
FROM Product;

-------------------------------------------------
-- 115. Assign row numbers to customers.
-------------------------------------------------
SELECT Customer_Name,
       ROW_NUMBER() OVER(ORDER BY Customer_Name) AS Row_No
FROM Customer;

-------------------------------------------------
-- 116. Insert a new customer.
-------------------------------------------------
INSERT INTO Customer
VALUES (111,'Raj','Male','Hyderabad','9876543210','Gold',600);

-------------------------------------------------
-- 117. Update customer city.
-------------------------------------------------
UPDATE Customer
SET City='Bengaluru'
WHERE Customer_ID=111;

-------------------------------------------------
-- 118. Delete customer.
-------------------------------------------------
DELETE FROM Customer
WHERE Customer_ID=111;

-------------------------------------------------
-- 119. Create Savepoint.
-------------------------------------------------
SAVEPOINT Before_Update;

-------------------------------------------------
-- 120. Rollback to Savepoint.
-------------------------------------------------
ROLLBACK TO Before_Update;



-- ============================================================
-- BUSINESS SCENARIO QUERIES (121-150)
-- Retail Sales Analytics
-- Oracle SQL
-- ============================================================

-------------------------------------------------
-- 121. Display customer with highest total purchase.
-------------------------------------------------
SELECT C.Customer_Name,
       SUM(O.Total_Amount) AS Total_Purchase
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
```

```
GROUP BY C.Customer_Name
ORDER BY Total_Purchase DESC;

-------------------------------------------------
-- 122. Display top 5 highest priced products.
-------------------------------------------------
SELECT *
FROM (
    SELECT Product_Name,
           Price
    FROM Product
    ORDER BY Price DESC
)
WHERE ROWNUM <= 5;

-------------------------------------------------
-- 123. Display department with highest average salary.
-------------------------------------------------
SELECT Department,
       AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department
ORDER BY Average_Salary DESC;

-------------------------------------------------
-- 124. Display most used payment method.
-------------------------------------------------
SELECT Payment_Method,
       COUNT(*) AS Total
FROM Payment
GROUP BY Payment_Method
ORDER BY Total DESC;

-------------------------------------------------
-- 125. Display total refund amount.
-------------------------------------------------
SELECT SUM(Refund_Amount) AS Total_Refund
FROM Return_Order;

-------------------------------------------------
-- 126. Display returned products.
-------------------------------------------------
SELECT DISTINCT P.Product_Name
FROM Product P
JOIN Return_Order R
ON P.Product_ID = R.Product_ID;

-------------------------------------------------
-- 127. Display customers who paid using UPI.
-------------------------------------------------
SELECT DISTINCT C.Customer_Name
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Payment P
ON O.Order_ID = P.Order_ID
WHERE P.Payment_Method='UPI';

-------------------------------------------------
-- 128. Display total sales by category.
-------------------------------------------------
SELECT C.Category_Name,
       SUM(OD.Total) AS Total_Sales
FROM Category C
JOIN Product P
ON C.Category_ID=P.Category_ID
JOIN Order_Details OD
ON P.Product_ID=OD.Product_ID
GROUP BY C.Category_Name;

-------------------------------------------------
-- 129. Display highest rated product.
-------------------------------------------------
SELECT *
FROM Product
WHERE Rating =
(
```

```
SELECT MAX(Rating)
FROM Product
);

-------------------------------------------------
-- 130. Display employee with highest sales.
-------------------------------------------------
SELECT E.Employee_Name,
       SUM(O.Total_Amount) AS Total_Sales
FROM Employee E
JOIN Orders O
ON E.Employee_ID=O.Employee_ID
GROUP BY E.Employee_Name
ORDER BY Total_Sales DESC;

-------------------------------------------------
-- 131. Display customers with points above 500.
-------------------------------------------------
SELECT *
FROM Customer
WHERE Points > 500;

-------------------------------------------------
-- 132. Display out of stock products.
-------------------------------------------------
SELECT *
FROM Product
WHERE Stock = 0;

-------------------------------------------------
-- 133. Display products with stock below 20.
-------------------------------------------------
SELECT *
FROM Product
WHERE Stock < 20;

-------------------------------------------------
-- 134. Display latest order.
-------------------------------------------------
SELECT *
FROM Orders
ORDER BY Order_Date DESC;

-------------------------------------------------
-- 135. Display earliest order.
-------------------------------------------------
SELECT *
FROM Orders
ORDER BY Order_Date ASC;

-------------------------------------------------
-- 136. Display average order amount.
-------------------------------------------------
SELECT AVG(Total_Amount) AS Average_Order
FROM Orders;

-------------------------------------------------
-- 137. Count successful payments.
-------------------------------------------------
SELECT COUNT(*) AS Successful_Payments
FROM Payment
WHERE Status='Success';

-------------------------------------------------
-- 138. Count pending payments.
-------------------------------------------------
SELECT COUNT(*) AS Pending_Payments
FROM Payment
WHERE Status='Pending';

-------------------------------------------------
-- 139. Display returned products with refund amount.
-------------------------------------------------
SELECT Product_ID,
       Refund_Amount
FROM Return_Order;
-------------------------------------------------
```

```sql
-- 140. Display customers from Bengaluru.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE City='Bengaluru';

-----------------------------------------------------
-- 141. Display employees hired before 2021.
-----------------------------------------------------
SELECT *
FROM Employee
WHERE Hire_Date < DATE '2021-01-01';

-----------------------------------------------------
-- 142. Display products priced above average.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Price >
(
SELECT AVG(Price)
FROM Product
);

-----------------------------------------------------
-- 143. Display total orders handled by each employee.
-----------------------------------------------------
SELECT Employee_ID,
       COUNT(*) AS Total_Orders
FROM Orders
GROUP BY Employee_ID;

-----------------------------------------------------
-- 144. Display highest order amount.
-----------------------------------------------------
SELECT MAX(Total_Amount) AS Highest_Order
FROM Orders;

-----------------------------------------------------
-- 145. Display lowest order amount.
-----------------------------------------------------
SELECT MIN(Total_Amount) AS Lowest_Order
FROM Orders;

-----------------------------------------------------
-- 146. Display Silver members.
-----------------------------------------------------
SELECT *
FROM Customer
WHERE Member_Type='Silver';

-----------------------------------------------------
-- 147. Display average product rating.
-----------------------------------------------------
SELECT AVG(Rating) AS Average_Rating
FROM Product;

-----------------------------------------------------
-- 148. Count returned orders.
-----------------------------------------------------
SELECT COUNT(*) AS Total_Returns
FROM Return_Order;

-----------------------------------------------------
-- 149. Display products with rating above 4.5.
-----------------------------------------------------
SELECT *
FROM Product
WHERE Rating > 4.5;

-----------------------------------------------------
-- 150. Display customer name, product name,
-- quantity ordered and total amount.
-----------------------------------------------------
SELECT C.Customer_Name,
       P.Product_Name,
       OD.Quantity,
```

```
       OD.Total
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Order_Details OD
ON O.Order_ID = OD.Order_ID
JOIN Product P
ON OD.Product_ID = P.Product_ID;
```